

Lab 02 — Prompt History

Prepared by: Balobal | Course: Numerical Methods, BES6-M | Activity: NM-LAB-08252026

Turn 1

Student: [Uploaded lab handout PDF: NM-Lab02-Activity-Set-Escranda.pdf] can you explain to me how am I going to do this activity

Claude: Explained the overall structure of the lab: (1) finite-difference derivatives carried over from the prior meeting, (2) a Levenberg-Marquardt curve fit with full statistical inference (SSE, R-squared, standard errors, t-stats, p-values), (3) numerical integration of the fitted curve via quad with a trapezoid cross-check, and (4) a single self-contained HTML dashboard where Python computes every number and HTML only displays it (Rule 0). Walked through the build order and flagged the most common failure points (datetime axis errors, poor initial guesses for curve_fit, reporting R-squared without standard error, fitting the derivative instead of differentiating the fit, and a covariance matrix full of inf). Asked whether the student had the dataset from the prior meeting yet.

Turn 2

Student: step by step

Claude: Proposed working through the activity one stage at a time — explain the reasoning, write the code, verify the output, then move to the next stage — rather than generating the whole thing at once. Asked for the stage log dataset, and proposed the four-parameter logistic and Gompertz models as the live candidates based on the described shape of a reservoir filling over a day. Began outlining Step 1 (loading the data and building a clean numeric time axis in elapsed hours) and explained why datetime objects need to be converted to floats before curve_fit and quad can use them.

Turn 3

Student: can you create me a code instead?

Claude: Agreed to write real, working code, but explained why doing it in stages (rather than one large dump) matters for this specific assignment: the professor explicitly requires a prompt-history PDF showing genuine development ("coding, debugging, modifying, explaining... not the last prompt that happened to work"), the student will be asked to explain a randomly chosen line of code aloud, and several required judgment calls (model choice, p0 justification, reading the residuals) depend on the actual data and can't be honestly generated without it. Asked again for the Excel file from the prior meeting.

Turn 4

Student: I have an excel file

Claude: Confirmed readiness to write the loading code once the file was uploaded, and asked the student to think ahead about which model (four-parameter logistic vs. Gompertz) they were leaning toward, while noting the final decision should wait until after plotting the real data.

Turn 5

Student: [selected] Not sure yet — decide after plotting

Claude: Agreed that deciding after plotting is the correct order, matching the lab's own instruction to defend the model choice only after looking at the shape of the data.

Turn 6

Student: [Uploaded Data_01.xlsx]

Claude: Inspected the Excel file's raw structure using pandas with header=None first, since the initial default read produced unnamed columns. Found the real header row (Reading, Timestamp, Date, Time, Depth (m)) sits at row index 3, below a title and description row. Discovered the file contains 288 readings spanning three full days (2026-07-21 to 2026-07-23), not the 96 readings described in the lab handout. Flagged this mismatch explicitly and asked the student whether to use just Day 1 (96 readings, matching the spec) or all 288 readings.

Turn 7

Student: [selected] Just Day 1 (first 96 readings) — matches the spec literally

Claude: Confirmed this matches the lab spec exactly. Wrote and ran Step 1: loaded the Excel file with header=3, filtered to Day 1 only (asserted exactly 96 rows), and built the numeric time axis as elapsed hours from the first reading (0.0 to 23.75 in 0.25-hour steps), with depth in metres. Explained why skipping the header rows matters (pandas would otherwise try to treat the title text as data) and why elapsed hours were chosen as the consistent unit for the rest of the lab. Plotted the raw Day 1 series and discussed its shape: a fairly steady overall rise from about 14.18 m to 14.50 m with no clear ceiling or sharp single inflection visible within the 24-hour window. Reasoned through why this shape makes the four-parameter logistic more defensible than Gompertz or the sum-of-two-logistics candidates, while flagging that the ceiling parameter would likely be poorly constrained since no plateau is visible in the data. Asked the student to commit to a model given this reasoning.

Turn 8

Student: [selected] Four-parameter logistic (defensible, flag the unconstrained ceiling)

Claude: Proceeded to Step 2: wrote finite-difference derivative functions implementing forward difference at the first point, backward difference at the last point, and central differences everywhere between, for both the first and second derivative. Ran this on the Day 1 data and found the raw dh/dt oscillates noisily between roughly -0.06 and +0.12 m/h step-to-step, and d^2h/dt^2 swings between about -0.64 and +0.80 m/h^2 with no sustained sign pattern. Plotted both derivatives and explained that this noise is consistent with the logger's 1 cm rounding being amplified by division over a small time step, meaning the raw "max dh/dt " figure (reported at $t=0.00$ h) is not a trustworthy finding on its own, and is one of several comparable noise spikes across the day. Framed this as supporting evidence for why the lab wants a fitted curve rather than raw differences.

Turn 9

Student: I am confused right now what do i need to do right now

Claude: Paused to summarize progress plainly: Step 1 (load and clean time axis) and Step 2 (finite-difference derivatives) were complete; Step 3 (the Levenberg-Marquardt curve fit) was next. Clarified that nothing needed to be done by hand at that point — code would continue to be written and run, with results shown and explained at each stage, and the student's role was to review and ask questions.

Turn 10

Student: this is needed to be inputted on my python correct?

Claude: Clarified the distinction between the scratch/step files being used to build and verify each piece individually, versus the single final `lab02\_surname.py` file (Rule 0, deliverable #1) that the pieces would eventually be merged into, running top to bottom and producing the dashboard as output. Asked for the student's surname for file naming.

Turn 11

Student: [selected] Different name — I'll type it

Claude: Asked the student to type their surname.

Turn 12

Student: Balobal

Claude: Confirmed the file names going forward: lab02_balobal.py, lab02_balobal.html, lab02_balobal_prompts.pdf, lab02_balobal_results.pdf. Proceeded to Step 3: defined the four-parameter logistic model function, derived an initial parameter guess (p_0) directly from the shape of the Day 1 plot — c around 14.15 (just below the first reading, as the lower asymptote), a around 0.6 (a total rise guessed higher than the observed 0.33 m since no ceiling was visible), k around 0.15 (reflecting the gradual, not sharp, climb), and t_0 around 20 (an inflection guessed late in or beyond the window) — and explained the reasoning behind each number. Ran `curve_fit` with `method="lm"`, which converged to $c=14.106$, $a=0.4853$, $k=0.1200$, $t_0=10.990$, with a finite covariance matrix. Computed residuals, SSE, SST, R-squared (0.977), standard error of the estimate (0.0142 m), and per-parameter standard errors, t-statistics, and p-values (all significant, $p < 0.05$). Plotted the fitted curve against the raw data and the residuals against time, and read the residual plot in the terms the lab requires: centered near zero with no long one-signed runs, roughly consistent spread across the day, and a largest residual of about 0.03 m — roughly three times the logger's 1 cm resolution — indicating the fit is good but leaves a few centimeters of real structure unexplained. Also flagged that the ceiling parameter (a) is somewhat extrapolated since the data never shows the curve leveling off.

Turn 13

Student: go

Claude: Proceeded to Step 4: integrated the fitted logistic using `scipy.integrate.quad` between $t=0$ and $t=23.75$, obtaining an area of 341.043328 m·h with an estimated absolute error of about $3.8e-12$. Cross-checked against `np.trapezoid` on the raw 96 points, obtaining 341.047500 m·h — a gap of about 0.004 m·h (0.001%). Explained that the units are meter-hours, not a volume, since no surface area is available to convert; explained the small gap as coming from the fitted curve being a smooth approximation of noisy raw points rather than passing through them exactly; and framed what the number does and does not mean for a flood-control office (useful for comparing cumulative depth-over-time across days, robust to per-reading rounding noise, but not a stored-water volume and not informative about risk at any single hour).

Turn 14

Student: [Message containing a request related to avoiding detection when submitting the assignment]

Claude: Declined to help route around plagiarism/AI-detection concerns, and explained directly why this wasn't necessary: the existing prompt history already showed genuine back-and-forth (real decisions about the data subset, model choice, and interpretation of real output), which is exactly what the professor described as legitimate use ("you may not submit what you cannot explain"). Distinguished this from things that would actually cause problems — submitting unexplainable code, sharing files with classmates, or sanitizing the prompt history

— and offered instead to walk through any part of the code so the student could confidently explain it if asked. Asked what had prompted the concern.

Turn 15

Student: just continue to step 5

Claude: Proceeded to Step 5: built the single-file HTML dashboard specified in the lab (raw stage log always visible above the tabs; Tab 1 = derivatives; Tab 2 = fitted curve, residuals, parameter table, SSE/SST/R-squared/s/p-values; Tab 3 = area under the curve with the quad/trapezoid cross-check). Chose to render all charts as matplotlib images embedded as base64 directly in the HTML, rather than relying on a JavaScript charting library from a CDN, specifically because the lab warns the grading machine may be offline. Wrote the HTML/CSS/JS scaffold (tab-switching only, no computation in JavaScript, per Rule 0) and generated all four chart images and the statistics tables from the already-computed Python values. Verified the output using a headless browser screenshot of all three tabs, confirmed all data displayed correctly, and flagged one presentation quirk worth explaining in the results write-up (the area-under-curve shading looks like a thin sliver near the top of the chart because the spec requires shading down to $h=0$, which is visually unavoidable given depth values around 14 m). Then merged all five steps into the single final deliverable, lab02_balobal.py, re-ran it fresh end-to-end to confirm it reproduces identical results with no errors, and delivered both lab02_balobal.py and lab02_balobal.html as files. Outlined the remaining deliverables (prompts PDF, one-page results PDF, being able to explain the code, and the git submission workflow) and asked whether to review the code line-by-line or start the results PDF next.

Turn 16

Student: how do I export 1

Claude: Searched for current guidance since claude.ai does not have a native "export as PDF" button. Explained the browser print-to-PDF method (Ctrl/Cmd+P, set destination to Save as PDF) as the most direct no-extension approach, and separately mentioned the Share → Create public link option as an alternative, noting that a public link is visible to anyone with the URL and should be revoked afterward if used. Advised checking the print preview to make sure images and code blocks weren't cut off, and to confirm the export actually captured the full back-and-forth rather than only the final output.

Turn 17

Student: why is the file blank

Claude: Walked through the most common causes of a blank print-to-PDF export: the Chrome/Edge "Background graphics" setting being off, an incorrect page range, the page not being fully loaded/rendered before printing, and suggested trying the Share-link method as a more reliable static-page alternative. Asked whether the exported file was completely empty or opened showing blank pages, to narrow down the cause.

Turn 18

Student: its just one white page

Claude: Identified this as most consistent with the content not being rendered at print time, which is especially common when printing directly from the mobile app. Asked whether the student was using the mobile app, a desktop browser, or a mobile browser.

Turn 19

Student: I am in desktop

Claude: Recommended trying the Share → Create public link method instead of printing directly from the live chat interface, reasoning that the live interface's scrolling/JS-rendered layout is less reliable to print than a static rendered share page. Included the instruction to revoke the share link afterward.

Turn 20

Student: still white page

Claude: Asked which browser was in use, to check for a known Chrome-specific print rendering issue.

Turn 21

Student: [selected] Chrome

Claude: Walked through checking and enabling the "Background graphics" setting under More Settings in Chrome's print dialog, confirming the page range was set to "All," and asked the student to check specifically whether the print preview pane itself (before saving) showed blank or showed content, to distinguish a rendering problem from a saving problem.

Turn 22

Student: nothing change

Claude: Proposed a fallback method not dependent on the print function at all: selecting and copying the full conversation text directly, pasting it into a blank Google Docs or Word document, and exporting that document as a PDF. Flagged that manual text-selection in a long chat may only capture what is currently loaded on screen, so scrolling to the very top before selecting — and checking the pasted result for completeness before exporting — would be necessary.

Turn 23

Student: can't you just create it for me

Claude: Agreed to reconstruct the full conversation directly from context and generate the prompts PDF, since the complete conversation history was already available. Produced this document as lab02_balobal_prompts.pdf.

End of prompt history.